

Fiche de révision — Panorama du Cloud & Architecture Azure (TP1)

Cours magistral « *Panorama du Cloud et Architecture Azure* » — Bloc 4, Mastère Dev Manager Full Stack. Cas fil rouge : conception d'une architecture cloud pour **ShopEasy** (application de gestion de commandes).

1. Du SI traditionnel au cloud

SI traditionnel : l'entreprise achète/loue des serveurs physiques, les installe, configure réseau et OS, gère sauvegardes, correctifs et capacité. Limites : délai de mise à disposition, **surdimensionnement** (acheter pour le pic), coût initial lourd, maintenance à sa charge, **point de défaillance unique** (mono-serveur).

Le cloud fournit des ressources **à la demande** (calcul, stockage, réseau, bases, sécurité, supervision...) via console, API ou IaC. Le vrai changement n'est pas que technique : on peut **tester vite, déployer automatiquement, mesurer finement les coûts et adapter la capacité**.

Cas entreprise. ShopEasy tourne sur un seul serveur (Apache + PHP + MySQL local). Panne disque = appli indisponible et base potentiellement perdue. Le cloud permet de séparer les couches, redonder, sauvegarder automatiquement et superviser — ce que le mono-serveur ne permet pas.

2. Les 4 propriétés fondamentales du cloud

Propriété	Définition	Point d'attention
Élasticité	Augmenter/diminuer les ressources selon le besoin	Peu de trafic la nuit, beaucoup en campagne promo
Facturation à l'usage	CAPEX → OPEX, on paie ce qu'on consomme	⚠ une ressource oubliée continue de coûter
Automatisation	Créer/reproduire/contrôler par code	Ne pas se limiter aux clics dans la console
Services managés	Le fournisseur prend en charge une partie de l'admin	Transforme la responsabilité, ne la supprime pas

Cas entreprise. Un site e-commerce passe de 100 à 10 000 visiteurs pendant le Black Friday : l'élasticité ajoute des instances le temps du pic puis les retire. En on-premise, il aurait fallu acheter (et payer toute l'année) du matériel dimensionné pour ce pic ponctuel.

3. IaaS / PaaS / SaaS

Plus le service est **managé**, plus le fournisseur prend de responsabilités ; plus il est bas niveau, plus on garde de **contrôle** (et d'administration).

Modèle	Le client gère...	Exemples Azure
IaaS	OS, correctifs, runtime, app, données, sécurité	Virtual Machines, Virtual Network
PaaS	Code, config, données, droits	App Service, Azure SQL Database
SaaS	Utilisateurs, données, config fonctionnelle	Microsoft 365, Dynamics 365

Le choix est un arbitrage entre contrôle, responsabilité, rapidité de déploiement, compétences, coût opérationnel et conformité.

Cas entreprise. Pour migrer vite une app existante sans la réécrire → **IaaS** (VM, proche de l'existant). Pour réduire l'administration système d'une app web standard → **PaaS** (App Service). Le même besoin « héberger une app web » a deux réponses selon le curseur contrôle/charge.

4. Concepts Azure de base

Hiérarchie : Tenant (annuaire d'identité Entra ID) → **Subscription** (facturation + gouvernance) → **Resource Group** (regroupe les ressources d'un projet/env) → **Resource** (VM, VNet, Storage...).

⚠ **Erreur fréquente** : tout mettre dans un seul resource group. Un RG doit traduire une **logique de cycle de vie** (application, environnement, projet).

Région vs Availability Zone : - **Région** = zone géographique (enjeux de latence, conformité, résidence des données, services disponibles). - **Availability Zone (AZ)** = datacenters **physiquement séparés** dans une région (alimentation/réseau indépendants) → résilience contre la panne d'un datacenter.

Cas entreprise. Une appli critique déploie ses VM sur **3 AZ** d'une même région : si un datacenter tombe, le service continue sur les deux autres. Le choix de la région, lui, peut être imposé par le RGPD (données qui doivent rester en Europe).

5. Azure Well-Architected Framework (les 5 piliers)

Cadre pour **évaluer et justifier** une architecture.

Pilier	Questions clés
Reliability	Que se passe-t-il en cas de panne ? Sauvegarde/reprise ?
Security	Accès limités ? Données protégées ? Flux filtrés ? Audit possible ?
Cost Optimization	Bon dimensionnement ? Coûts suivis ? Environnements inutiles arrêtés ?
Operational Excellence	Exploitation standardisée ? Changements tracés ? Supervision ?

Pilier	Questions clés
Performance Efficiency	Ressources adaptées ? Architecture capable d'absorber la croissance ?

Cas entreprise. Avant de valider une archi en comité, l'architecte la passe au crible des 5 piliers. Une archi ShopEasy mono-VM échoue sur *Reliability* (SPOF) et *Operational Excellence* (pas de supervision) → on impose segmentation réseau, base managée, monitoring et règles de sécurité.

6. Réseau Azure : VNet, subnets, NSG

- **Virtual Network (VNet)** = réseau privé isolé (ex. **10.10.0.0/16**), équivalent d'un réseau local virtuel.
- **Subnets** = subdivisions par **fonction / niveau d'exposition** : public, applicatif (web), données, administration (bastion).
- **NSG (Network Security Group)** = règles de filtrage (priorité, source, destination, protocole, port, autoriser/refuser), associées à un subnet ou une interface.

Règles recommandées pour le TP :

Flux	Port	Source	Justification
HTTP	80	Internet / LB	Accès appli web de test
HTTPS	443	Internet / App Gateway	Chiffrement (cible prod)
SSH	22	IP admin uniquement	Administration contrôlée
SQL	1433	Subnet applicatif	Empêcher l'exposition directe de la base

⚠ Ouvrir SSH à **0.0.0.0/0** = mauvaise pratique → limiter à l'IP admin ou **Azure Bastion**.

Cas entreprise. Segmenter web / données / admin permet d'appliquer des règles différentes par couche et de limiter les **mouvements latéraux** : même si le serveur web est compromis, l'attaquant n'atteint pas directement la base (subnet data filtré).

7. Calcul : VM vs App Service

Azure Virtual Machines (IaaS) : on choisit image, taille, disque, réseau. Utile pour comprendre l'IaaS, **migrer une app existante** ou garder un fort contrôle. Le **dimensionnement** doit équilibrer CPU/mémoire/disque/réseau/budget (mauvais dimensionnement = mauvaise perf **ou** gaspillage).

Azure App Service (PaaS) : plateforme managée pour apps web, réduit l'administration OS/patches.

Critère	Virtual Machines	App Service
Contrôle système	Élevé (OS, services)	Limité (managé)
Administration	Lourde (patches, durcissement)	Simplifiée (focus code)

Critère	Virtual Machines	App Service
Migration existante	Adaptée	Peut demander une adaptation

Cas entreprise. Une grosse VM unique simplifie l'exploitation au début **mais concentre le risque**. Plusieurs petites VM derrière un Load Balancer améliorent la disponibilité et préparent la **scalabilité horizontale**.

8. Équilibrage de charge & disponibilité

Un **Load Balancer** répartit le trafic entre plusieurs instances : si une VM ne répond plus aux **sondes de santé**, le trafic part vers une autre.

Critère	Azure Load Balancer	Application Gateway
Niveau	Couche 4 (TCP/UDP)	Couche 7 (HTTP/HTTPS)
Usage	Répartition réseau simple	Routage web, terminaison TLS, WAF
Cas TP	Suffisant (1re archi)	Cible web professionnelle

⚠ Un Load Balancer **ne rend pas** une app automatiquement hautement disponible : la HA = **plusieurs instances + plusieurs zones + sondes + sauvegardes + supervision + conception adaptée**.

Cas entreprise. Pour une appli web exposée qui a besoin de chiffrement TLS, de routage par URL et d'un pare-feu applicatif (WAF), on choisit **Application Gateway**. Pour répartir un flux TCP simple entre 2 backends, **Load Balancer** suffit.

9. Stockage Azure

Storage Account = compte de stockage multi-usages. Pour une app web, le cas courant est le **Blob Storage** (objet) : images, documents, exports, archives.

Service	Usage	Exemple ShopEasy
Blob Storage	Objets / fichiers non structurés	Factures PDF, images produits
Managed Disks	Disques attachés aux VM	Disque système d'une VM
Azure Files	Partage de fichiers SMB/NFS managé	Partage entre applications

Bonnes pratiques : activer le **versioning** si restauration nécessaire, **éviter les conteneurs publics** (sauf justification), **politiques de cycle de vie** (archiver/supprimer les anciens), tags + nommage, surveiller les coûts (volume, transactions, rétention).

Cas entreprise. Externaliser les documents clients du disque local de la VM vers du Blob privé = durabilité (réplication), accès indépendant des VM, et **cycle de vie** (déplacer en tier Archive après

90 jours) pour réduire les coûts.

10. Base de données : Azure SQL Database vs SQL sur VM

Critère	SQL sur VM (IaaS)	Azure SQL Database (PaaS)
Contrôle	Très fort (OS + moteur)	Limité, managé
Maintenance	À la charge du client	Largement simplifiée
Sauvegardes	À concevoir et opérer	Intégrées
Disponibilité	À construire (cluster, réplication)	Options managées
Cas TP1	Comprendre l'héritage on-premise	Recommandé pour une cible cloud

Points de vigilance : ne pas exposer la base à Internet, stratégie de sauvegarde/restauration, niveau de service adapté, surveiller CPU/DTU-vCore/connexions/stockage, **moins privilège** sur les comptes applicatifs.

Cas entreprise. Migrer la base MySQL/SQL locale de ShopEasy vers **Azure SQL Database** décharge l'équipe des patches, sauvegardes et HA → elle se concentre sur les données, pas sur l'administration du moteur.

11. Identité, droits & gouvernance

Microsoft Entra ID fournit les identités/authentification. **Azure RBAC** accorde des droits sur des **scopes** (management group → subscription → resource group → resource). Principe de base : **le minimum de droits, pour la durée nécessaire** (moins privilège).

Rôle	Usage
Owner	Admin complète + gestion des droits → à limiter fortement
Contributor	Gère les ressources, pas les droits → à encadrer
Reader	Consultation seule → audit, observation
Cost Management Reader	Consultation des coûts → FinOps

⚠ Donner **Owner à toute l'équipe** est une erreur de gouvernance.

Cas entreprise. Un compte admin **partagé** (cas ShopEasy actuel) = perte de traçabilité + un seul secret compromis = accès total. La cible : comptes **nominatifs** + RBAC + MFA → on sait qui a fait quoi, et chacun n'a que ses droits.

12. Supervision : Azure Monitor

Une archi non supervisée est ingérable. Azure Monitor collecte **métriques** et **logs** et déclenche des **alertes**. Il répond à 3 questions : *ça marche ? pourquoi pas ? comment ça évolue ?*

Ressource	Métriques utiles
VM	CPU, mémoire, disque, réseau
Load Balancer / App Gateway	Requêtes, erreurs, santé des backends
Azure SQL	CPU, stockage, connexions, latence
Storage Account	Transactions, capacité, erreurs

Une bonne alerte = symptôme + seuil + cible + criticité + action attendue. Trop d'alertes = bruit ; trop peu = incidents masqués.

Cas entreprise. Alerte « CPU VM > 80 % pendant 5 min » → l'équipe est prévenue avant que les utilisateurs ne subissent des lenteurs, et peut scaler ou diagnostiquer avant l'incident.

13. FinOps & maîtrise des coûts

Dans le cloud, le coût est un **critère d'architecture**, pas une question de fin de projet. Chaque choix (taille VM, stockage, rétention des logs, trafic sortant, HA, niveau SQL) a un impact financier.

Outils : Azure **Pricing Calculator** (estimer avant), **Cost Management** (suivre, analyser, budgets), **Tags** (ventiler par app/env/équipe), **budgets + alertes**.

Situation	Action FinOps
VM de test allumées H24	Arrêt programmé, suppression des ressources obsolètes
Logs conservés trop longtemps	Politique de rétention adaptée
Base SQL surdimensionnée	Ajustement du niveau selon métriques
Absence de tags	Convention de tagging obligatoire

Cas entreprise. *Réduire* les coûts = dépenser moins. *Optimiser* = meilleur rapport **coût/valeur** (bon dimensionnement, bon modèle de facturation, automatisation) sans dégrader le service. L'objectif FinOps est l'optimisation, pas le minimum à tout prix.

14. Sécurité cloud — modèle de responsabilité partagée

Le fournisseur sécurise l'**infrastructure physique** et une partie des services. Le client reste responsable de la **configuration, des identités, des données, des accès, des secrets** et de l'architecture.

Idée clé. Utiliser Azure **ne rend pas** une app automatiquement sécurisée. Une mauvaise config IAM, un stockage public ou un port d'admin ouvert introduisent des risques majeurs.

Risques typiques ShopEasy → **mesures** : SSH ouvert (→ IP admin/Bastion/MFA), base exposée (→ accès privé, règles strictes), droits trop larges (→ RBAC/moindre privilège), stockage public (→ accès privé), absence de logs (→ Monitor/journaux/alertes).

Cas entreprise. Un Storage Account laissé en accès public « pour tester » expose les documents clients à tout Internet → fuite de données personnelles et risque RGPD. En prod : accès **privé**, SAS ou identité managée.

15. Méthode de choix des services & lecture critique

Partir du besoin, pas du service : qui utilise l'app ? quelles données ? quelle disponibilité attendue ? quelles contraintes de sécurité ? quel budget ?

Matrice de décision (extrait) :

Besoin	Option simple	Option cible	Critère
Application web	VM	App Service ou VM + LB	Contrôle, migration
Base SQL	SQL sur VM	Azure SQL Database	Admin, sauvegarde, dispo
Documents	Disque VM	Storage Account	Durabilité, versioning
Accès admin	SSH public	Bastion / restriction IP	Sécurité, audit

Lecture critique d'une archi (savoir critiquer = identifier forces, risques, arbitrages) : les ressources exposées sont-elles identifiées ? la base est-elle protégée ? supervision ? coûts estimés/attribuables ? **point de défaillance unique** ? droits limités ? données sauvegardées/versionnées ? choix justifiés par le besoin métier ?

Cas entreprise. En revue d'architecture, un pair pose ces 8 questions. Si l'archi a un SPOF ou une base exposée, elle ne passe pas — peu importe qu'elle « fonctionne ».

16. Nommage & tags

Un nommage incohérent rend l'exploitation difficile. Convention type : **rg-application-env-region**, **vnet-...**, **snet-role-...**, **vm-role-num-env**. Storage Account : nom court unique en minuscules.

Tags recommandés : **Application**, **Environment** (Dev/Prod), **Owner**, **CostCenter**, **Criticality**.

Cas entreprise. Avec un nommage et des tags standardisés, on filtre instantanément « toutes les ressources prod de ShopEasy appartenant à l'équipe X » pour un audit ou une analyse de coûts.

Les réflexes à retenir pour le TP1

- 1. **Cloud ≠ bonne architecture automatique** : les choix techniques restent décisifs.
- 2. **IaaS/PaaS/SaaS = arbitrage** contrôle ↔ responsabilité ↔ charge d'admin.
- 3. **Segmenter** le réseau (web/data/admin) + NSG restrictifs.
- 4. **Jamais SSH 0.0.0.0/0**, jamais de base/stockage exposés.
- 5. **Supprimer le SPOF** : plusieurs instances + LB (+ zones).
- 6. **Moindre privilège** (RBAC) + comptes nominatifs + MFA.
- 7. **Superviser** (Monitor) et **estimer/suivre les coûts** (FinOps).
- 8. **Justifier chaque service** par un besoin métier (Well-Architected).

Glossaire express

Terme	Définition
IaaS / PaaS / SaaS	Niveaux d'abstraction du service cloud
Tenant / Subscription / RG	Identité / facturation / regroupement de ressources
Région / AZ	Zone géographique / datacenters séparés dans une région
VNet / Subnet / NSG	Réseau privé / subdivision / filtrage de flux
Load Balancer / App Gateway	Répartiteur couche 4 / couche 7 (TLS, WAF)
Storage Account / Blob	Compte de stockage / stockage objet
Azure SQL Database	Base SQL managée (PaaS)
Entra ID / RBAC	Identités / contrôle d'accès par rôle
Azure Monitor	Supervision (métriques, logs, alertes)
Well-Architected	Cadre d'évaluation (5 piliers)
FinOps	Pilotage et optimisation des coûts cloud

Note déploiement (Azure for Students). Région **francecentral** refusée par cet abonnement → utiliser **swedencentral**. Taille **Standard_B1s** indisponible → **Standard_B2ts_v2**.